

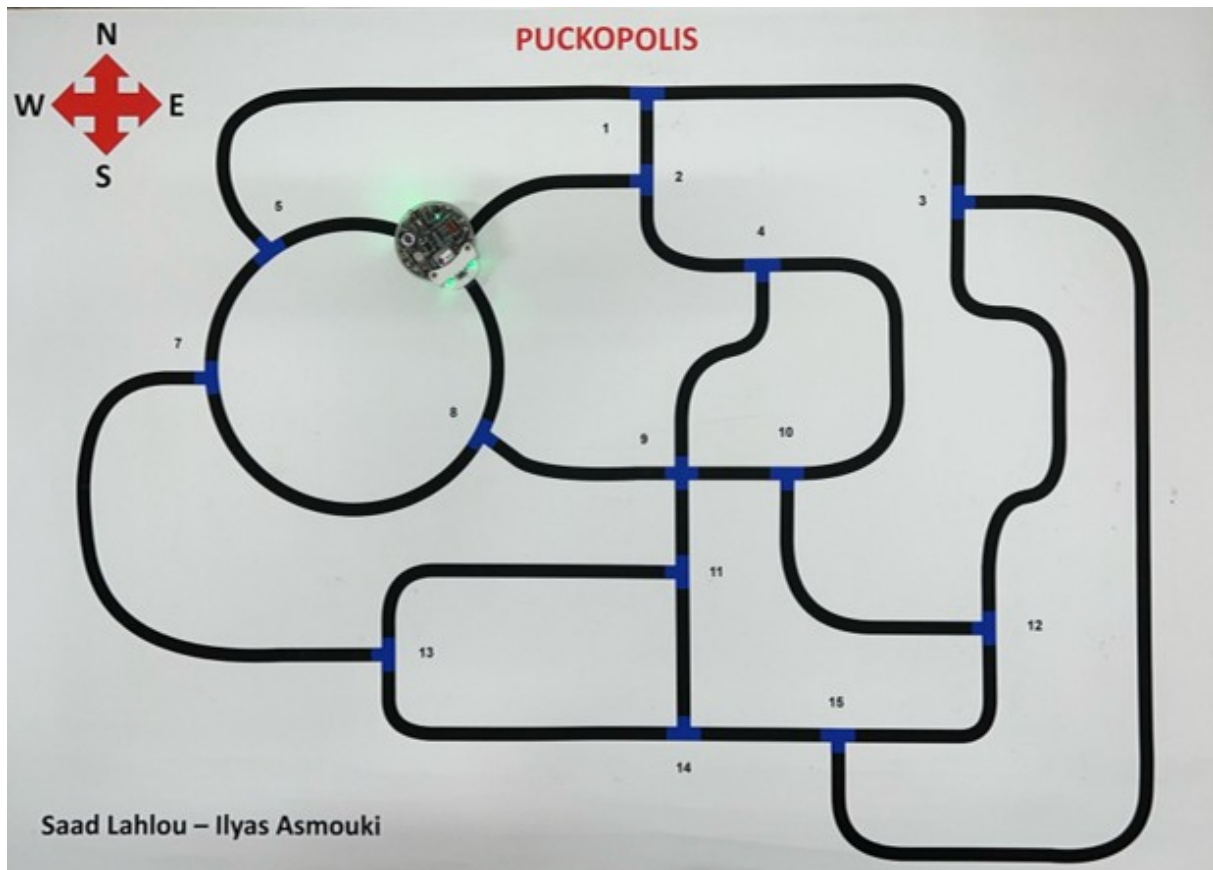
Puckopolis

Robotics Project Report

Course: Embedded Systems and Robotics (MICRO-315)

Professor: Dr. Francesco Mondada

Group 10: Saad Lahlou (361150), Ilyas Asmouki (356263)



July 5, 2025

Summary

Contents

1	Introduction	2
2	General Architecture	2
3	System Core Modules	3
3.1	Finite State Machine (FSM)	3
3.2	A* Path Finding	4
3.3	Color Detection	4
3.4	Obstacle Detection & Path Recomputing	5
3.5	Yaw Estimation & Path Correction	6
4	Team Collaboration and Organization	7

1 Introduction

The aim of this project is to develop a navigation system for the e-puck2 robot that finds the shortest path between two locations in a miniature city called “Puckopolis”. The robot accepts start and goal nodes, as well as an initial orientation via a GUI running on a host PC. It then follows a black line network to traverse the map, detecting blue markers at each waypoint. When the robot detects a red line on the ground, it stops and remains stationary until it detects a green line, at which point it resumes motion. This simulates red and green lights in traffic. If a physical obstacle obstructs its forward movement, the robot recalculates the shortest path around the blockage. In the sections that follow, we detail the general structure of the project and each module implemented to ensure reliable and robust operation.

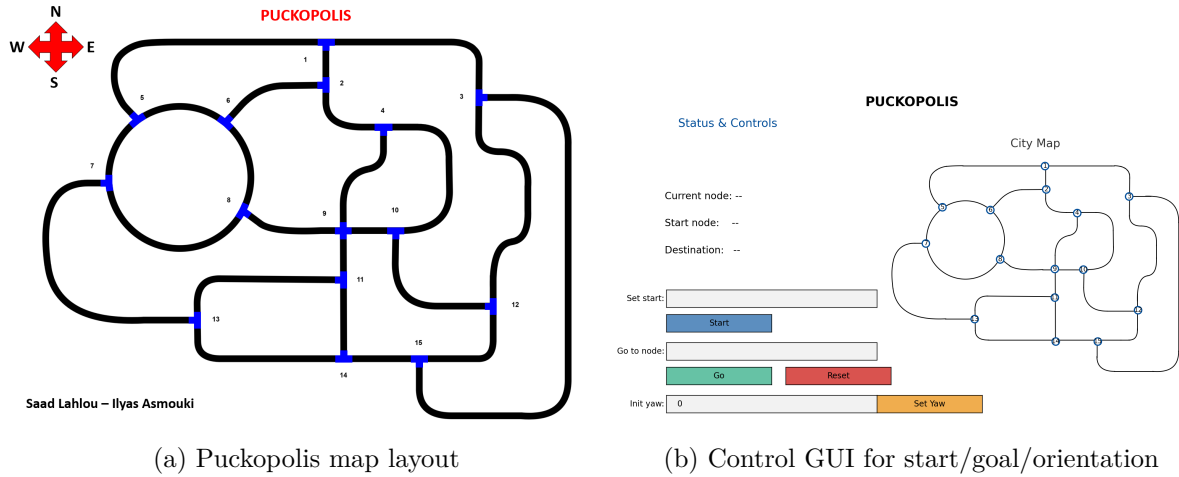


Figure 1: (a) Puckopolis map and (b) Graphical User Interface

2 General Architecture

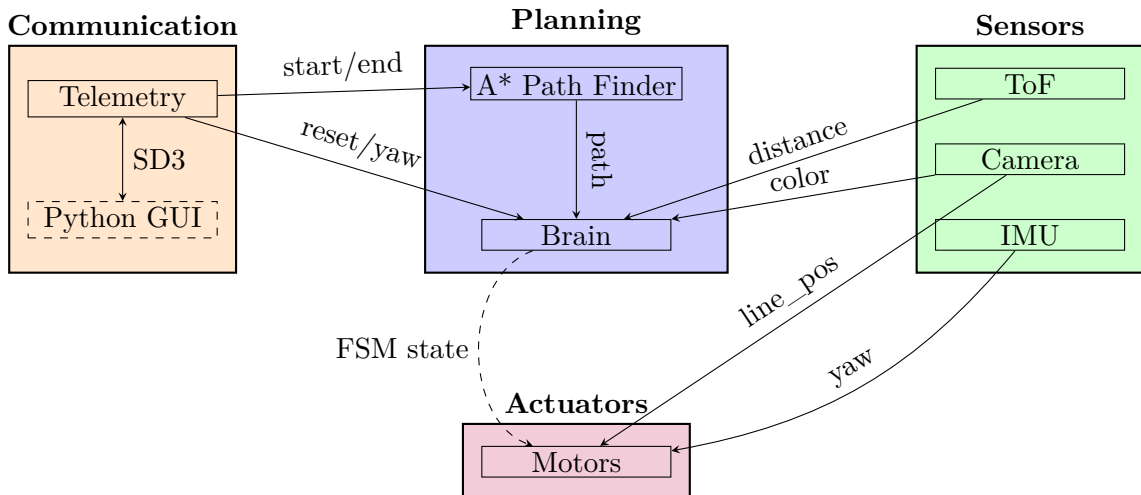


Figure 2: General Architecture

Our architecture is divided into four main modules. *Communication*, *Planning*, *Sensors*, and *Actuators*. Each are represented by a colored box in Figure 2. Within **Communication**, the Telemetry and Python GUI submodules exchange data over SD3 (USB or Bluetooth). In **Planning**, the A* Path Finder and FSM “Brain” coordinate path computation and mission logic.

The **Sensors** module comprises ToF, Camera, and IMU threads, while the **Actuators** module contains the Motor PID controller. All inter-module interactions use named topics on a message bus, except for two special channels: (1) the GUI→Planning link carrying **start/end/yaw/reset** signals over serial, and (2) the FSM state variable, stored as a static global in the Brain and accessed via a getter.

When a mission begins, the GUI sends **start**, **end**, and initial **yaw** to Telemetry, which immediately invokes the A* Path Finder with pointers to the FSM’s local graph and path arrays. Once A* computes a valid node sequence, it returns the path to the Brain and the FSM transitions from **READING** to **MISSION**.

During execution, the Camera thread continuously publishes two topics: **/color** to the Brain (for waypoint and stop logic) and **/line_pos** to the Motor PID loop. A dedicated obstacle-detection subroutine in the Brain subscribes to the ToF’s **/distance** topic, and the IMU thread streams filtered **yaw** directly into Motor PID for precise rotations. Depending on the current FSM state, the Motor PID loop either follows the black line or pauses for waypoint alignment. Turns are triggered by reading the static FSM state and sending a rotation command.

For modules requiring strict sequencing, we retain semaphore-based synchronizations between Camera’s capture and process threads, and between Telemetry and the FSM’s reset thread. Finally, we also drive an LED indicator that reflects the FSM’s current state.

3 System Core Modules

3.1 Finite State Machine (FSM)

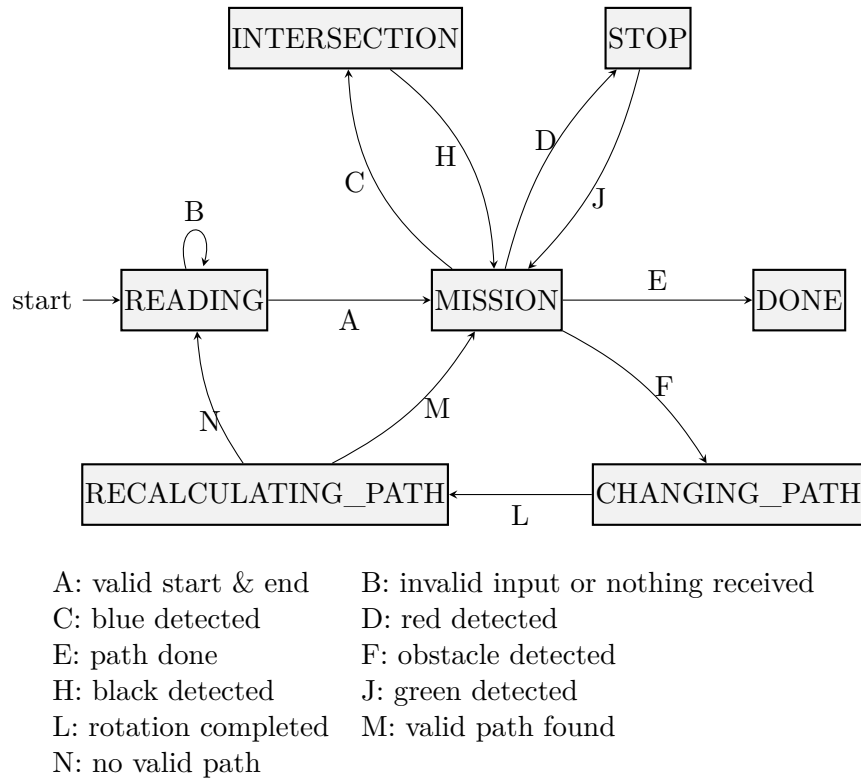


Figure 3: State Transition Diagram

Our e-puck2 driver is controlled by a state transition diagram driven by three event types: color detection (blue = waypoint arrival; red = stop; green = resume), obstacle sensing (triggering path recalculation), and path validation (success or failure). Additionally, the GUI’s Reset button forces the FSM back to the **READING** state from any current state.

The FSM has seven states:

- **READING:** await valid start/goal/orientation.
- **MISSION:** follow the black line and monitor sensors.
- **INTERSECTION:** handle detected waypoints.
- **STOP:** pause on red until green is seen.
- **RECALCULATING_PATH:** compute a new path that discards obstacles.
- **CHANGING_PATH:** perform any necessary rotation.
- **DONE:** route successfully completed.

3.2 A* Path Finding

To support A* without sacrificing memory on our embedded e-puck2, we organized three core data structures:

- **Node:** stores the A* costs g , h , and total $f = g + h$, plus a **parent** index to reconstruct the path.
- **Graph:** describes the city by its number of nodes N , and a fixed-size adjacency table `adjacency[N][M]` (where M = maximum neighbors). Each entry holds the neighbor index and an **angle** field indicating the heading the robot must adopt when traversing from node i to node j .
- **Path:** records the resulting sequence of node indices, its length, and total path cost.

By using a compact `[N][M]` array, we eliminate unused rows or columns and keep memory usage predictable. The per-edge angle lets the robot immediately align itself on the correct black line after each intersection.

Since we were unable to identify a suitable admissible heuristic, we set $h = 0$ for all nodes. This causes A* to behave identically to Dijkstra’s algorithm in our implementation.

3.3 Color Detection

We detect five surface colors, *black*, *green*, *red*, *blue*, and *white*, by analyzing pixel intensity profiles in the camera’s color buffers. For each buffer i , we first discard the first and last `INDEX_OFFSET` = 50 pixels (which typically contain spurious noise), then compute the intensity drop

$$\Delta_i = \max(I_i) - \min(I_i)$$

and compare it to a threshold

$$\tau_i = \mu_i \cdot \kappa_i,$$

where μ_i is the mean intensity and κ_i a tunable gain. A color is declared detected when $\Delta_i > \tau_i$. To reduce false positives, we apply majority-vote pooling over a sliding window of $W = 7$ frames, suppressing isolated misdetections.

Line-position extraction runs on every frame’s buffer analysis to supply the motor control thread with high-frequency updates. If the drop is absent in the buffer corresponding to the detected color (e.g. no drop in the red channel), the algorithm falls back to an alternate buffer (e.g. green) for reliable positioning. This ensures accurate and responsive line following even amid occasional misclassifications. Early in testing we only detected blue, green, red, and black,

but occasional rotations caused the e-puck to “see” white and misclassify, typically as green, producing erratic behavior. As a last-resort mitigation, we now detect white by evaluating all five buffers in parallel (with individually tuned thresholds) and, upon white detection, command the robot to rotate 20° opposite the last line-following error. The robot repeats this correction until the black line is recovered. With properly calibrated thresholds, white should never be encountered, making this fallback unnecessary in normal operation. But the high dependence on the environment makes this change necessary.

3.4 Obstacle Detection & Path Recomputing

To simulate a real taxi/cab driver navigating urban traffic, the robot has to be aware of its surroundings, and especially of whatever is ahead of it. This requires a forward-facing, responsive obstacle-detection mechanism that can trigger route replanning when necessary. For this purpose, we relied on the on-board Time of Flight sensor which provides millimeter-resolution distance measurements.

One of the main problems came across the first trials. Raw ToF readings were highly jumpy, and sometimes caused spurious signals that caused the robot to think something is ahead of it when it isn’t. At rest, the variance of the sensor is roughly $\sigma^2 = 8.52 \text{ mm}^2$ at around 3 to 5 cm, and increases non-linearly at higher distances. To address this, we implemented a lightweight Kalman filter to estimate the true distance by balancing trust between the sensor’s noisy measurements and the prior estimate. The Kalman filter assumes that sensor noise follows a Gaussian distribution. To verify this, we recorded over 300 static measurements from the ToF sensor at a fixed distance of 100 mm (there is an offset in the sensor’s readings defined as `DIST_OFFSET_MM` which we subtracted in software) and plotted the histogram of unfiltered values.

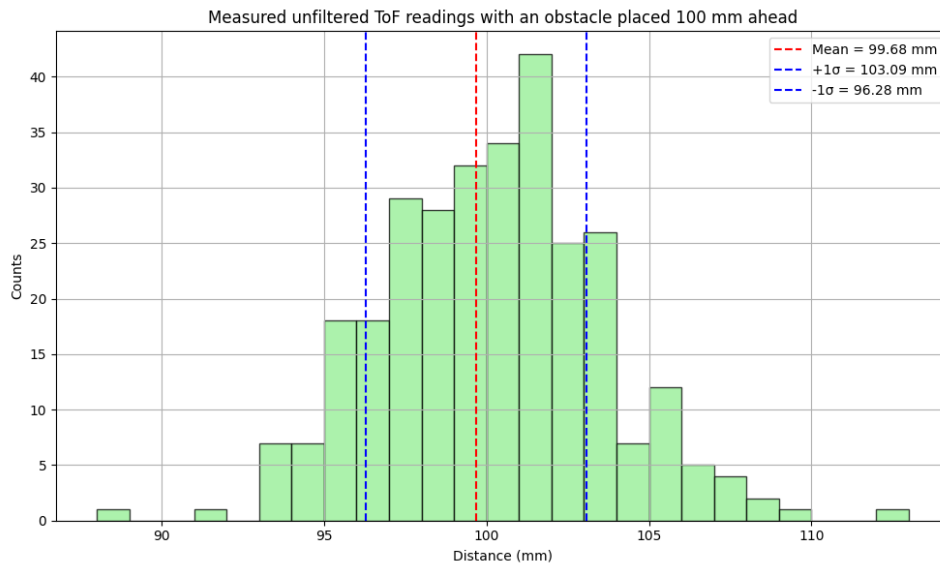


Figure 4: Histogram of raw ToF measurements (10 cm static target). The data roughly looks like a Gaussian distribution, justifying the use of Kalman filtering.

As shown in Figure 4, the raw sensor noise shows a bell-shaped curve, consistent with the Gaussian noise assumption. This validates the use of a Kalman filter to optimally estimate the true distance in the presence of such noise.

To evaluate the dynamic behavior of the filter, we moved an object from 10 cm to 5 cm and back to 10 cm, recording both raw and filtered values. As shown in Figure 5, the Kalman filter significantly reduces noise while closely tracking the true signal. The raw values exhibit high-frequency jitter and spurious spikes, which could lead to false detections in the FSM. In contrast,

the filtered response is smoother but lags the true signal a bit. We tuned the `PROC_NOISE` parameter to get performances that are good enough given our setup.

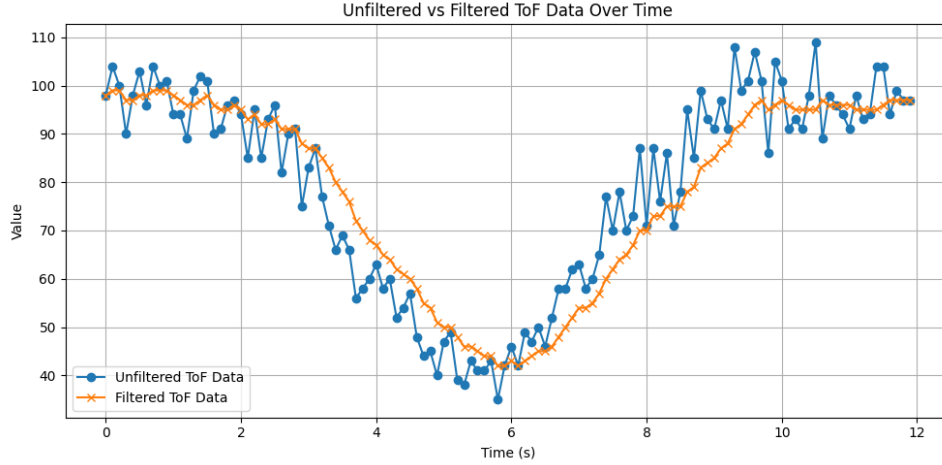


Figure 5: Comparison of raw (noisy) vs. Kalman-filtered ToF readings during motion (10 → 5 → 10 cm)

Obstacle Detection Logic

The filtered distance is published at 10 Hz on the `/distance` topic via a dedicated ToF thread. The FSM subscribes to this topic and monitors the latest distance reading. When an obstacle is detected within a predefined threshold (`OBSTACLE_THRESHOLD_MM`), the FSM transitions such that the robot halts and flashes its obstacle LEDs. It then performs a 180° turn to face away from the obstruction, then the edge leading into the obstacle is marked as “blocked” in the A* graph. The robot then resumes its mission as usual.

3.5 Yaw Estimation & Path Correction

The robot’s ability to correctly align itself on the map relies on knowing its yaw orientation with sufficient accuracy. Initially, we attempted multiple filtering strategies, including a full-state Extended Kalman Filters (EKF) modeling both translational and rotational motion (15 states: 3 linear translation, 3 linear velocity, 3 orientation, then 6 bias states for all 6 axes) but found that for our application, most of these approaches added unnecessary complexity without real performance benefits. The Kalman filter also turned out to be a lot harder to tune, with unconvincing results.

Ultimately, we settled on a minimal model that performs dead-reckoning of yaw by integrating the gyroscope’s angular rate g_z :

$$\theta_k = \theta_{k-1} + (g_z - b_z) \cdot \Delta t$$

Here, θ_k is the estimated yaw angle at time step k , g_z is the measured angular velocity around the vertical axis, b_z is a fixed bias (estimated to be 0), and Δt is the IMU update period (4 ms). To match existing project naming conventions from past experiments, this implementation is labeled as an “EKF”, but it is mathematically equivalent to a simple one-dimensional Kalman filter with constant system and measurement Jacobians. The true model is linear and comprises just two states:

$$\mathbf{x} = \begin{bmatrix} \theta \\ b_z \end{bmatrix}$$

The bias b_z is treated as a constant in practice, because its rate of change is negligible across the mission timescale. Although our Kalman implementation includes a process noise term to model

b_z drift (random walk), we found that this had no observable impact and could be omitted with no degradation.

Yaw Usage in Mission

The estimated yaw is published at 250 Hz on the topic `/imu_yaw` and used by the motor controller. Each time a waypoint is reached (blue line detected), the robot compares the current yaw estimate to the target heading required for the next A* path segment. It then rotates until the error falls below a set threshold. This ensures proper alignment before resuming motion.

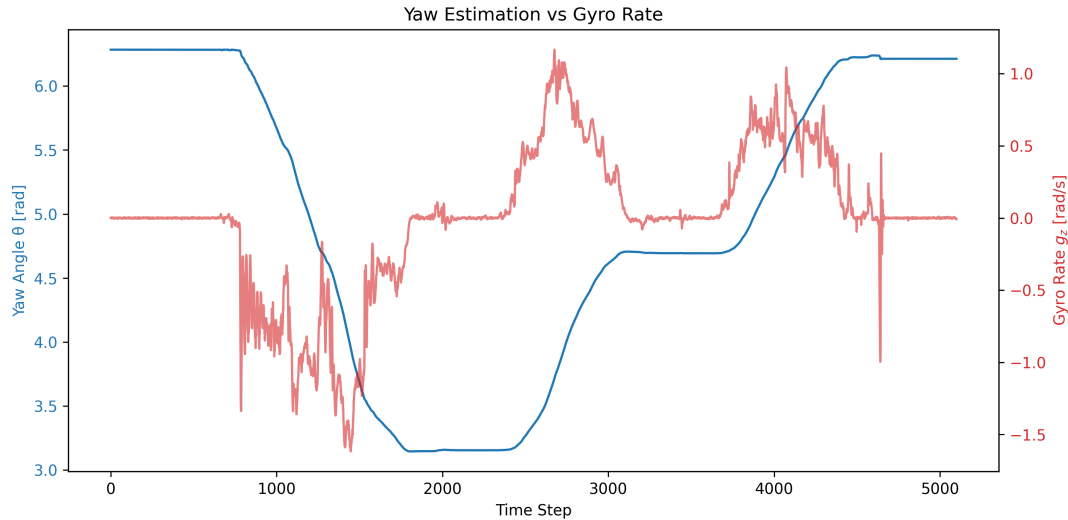


Figure 6: Yaw estimate over time during a 180° in-place rotation, then 90° in the opposite direction. The estimation is smooth and tracks the motion accurately.

In practice, this integrator provides reliable orientation tracking for missions up to 2-3 minutes. As shown in Figure 6, the yaw estimate closely matches the robot’s actual rotation trajectory.

Limitations. Our system assumes no slippage during rotation and uses no external correction source (e.g., magnetometer, motor or visual odometry). Therefore, in missions requiring absolute orientation consistency over long durations, this estimator would need to be fused with additional measurements or landmarks. For the city-scale Puckopolis map, however, our current approach proved sufficient.

4 Team Collaboration and Organization

Looking back at this project, we brainstormed many scenarios: maze navigation, city routing, and other nav-based projects. We then settled on Puckopolis, a miniature city requiring line following, waypoint stops, obstacle handling, and dynamic yaw alignment. With the project rules defined, we broke the system into logical modules (planning, sensing, communication, actuation) and designed the necessary threads and data structures. We paid particular attention to how the map and computed path would be stored in memory. We also identified potential pitfalls (sensor noise, heading errors, color-detection sensitivity) and incorporated strategies: Kalman-filtered IMU readings, threshold tuning, majority-vote pooling, to mitigate them before writing a single line of code.

Throughout development, we adopted a collaborative workflow: both team members programmed side by side whenever possible, each taking primary responsibility for specific fea-

tures yet reviewing and testing each other's work. We adopted a philosophy that maximized modularity, which made the work merging very easy. In Git, we maintained a main (RENDU_MINIPROJET_2025) branch protected by feature branches: each new capability lived on its own branch, which was merged into `RENDU_MINIPROJET_2025` only after review and testing. For visual design, particularly the city map, we used Draw.io, exporting our final layout as a reference for both the path-planning algorithm and the A* graph structure.

Conclusion

The development of Puckopolis successfully demonstrated the integration of multiple embedded systems concepts into an autonomous robot driver. Through careful modular design, robust sensor reading, and an efficient pathfinding strategy, the e-puck2 robot was able to navigate a miniature city with dynamic obstacle avoidance, precise waypoint handling, and real-time decision-making. Despite hardware constraints, we implemented a memory-efficient A* algorithm and lightweight filtering techniques that achieved reliable performance in a noisy physical environment.

Throughout the project, we emphasized modularity, testability, and clarity, both in software architecture and team collaboration. This allowed us to isolate and address key challenges such as sensor noise, heading drift, and ambiguous color detection. The final system not only performed its intended tasks but also remained flexible enough to accommodate extensions, such as traffic simulations or enhanced localization methods.

Looking forward, improvements such as long-term yaw drift correction (for example magnetometer fusion or visual odometry) and more adaptive obstacle handling would further increase the robustness and autonomy of the system. Nonetheless, Puckopolis stands as a complete and functional demonstration of reactive, sensor-driven navigation in embedded robotics.

References

- [1] F. Mondada, *Embedded Systems and Robotics (MICRO-315)*, EPFL, <https://edu.epfl.ch/coursebook/fr/systemes-embarques-et-robotique-MICRO-315>.
- [2] G. Welch and G. Bishop, “Kalman Filter Tutorial,” <https://www.kalmanfilter.net/default.aspx>.
- [3] “draw.io Diagramming,” <https://app.diagrams.net/>, used for city-map design.
- [4] “A* Search Algorithm,” Wikipedia, https://en.wikipedia.org/wiki/A*_search_algorithm.
- [5] “Kalman Filter,” Wikipedia, https://en.wikipedia.org/wiki/Kalman_filter.